

Laboratory Exercise 04

Topics

1. `while` and `for` loops
2. Processing complex inputs
3. Simulations

Discussion

- A. Discuss similarities and differences between the following two sets of statements:
 - a. `if`, `elif`, and `else`;
 - b. `while` and `for`.
- B. What is an infinite loop?
- C. What is a simulation?

Exercises (in red = suggested ones to complete during the lab)

Part 1 – Basic Cycles

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least two exercises during the lab session, and the remaining ones at home.

[SUGGESTED] 04.1.1 Integers. Write a program that reads a sequence of integers (the sequence ends when an empty string is inserted) and that, after each new acquisition, calculates and displays:

- I. the partial sums of all the numbers acquired; The program must display the result of the calculations after each new acquisition.
If, for example, the input values are `1 7 2 9`, the program will display the partial sum of the numbers acquired after each acquisition:
 - a. at the first acquisition (`1`), the first value acquired: `1`;
 - b. at the second acquisition (`7`), the sum of the first and second acquisitions: `1 + 7 = 8`;
 - c. at the third acquisition (`2`), the sum of the first, second and third acquisitions: `1 + 7 + 2 = 10`;
 - d. at the fourth acquisition (`9`), the sum of the first, second, third and fourth acquisitions: `1 + 7 + 2 + 9 = 19`.
- II. the minimum and maximum value among those acquired;
- III. the number of even values and the number of odd values between those captured. [P4.2]

[SUGGESTED] 04.1.2 Parsing a String. Write a program that reads a line of incoming text as a string and displays the following:

- I. only the uppercase letters of the string;
- II. letters in even positions in the string;
- III. the same string in which, instead of vowels (upper or lowercase), there is an underscore (`_`);
- IV. the number of numeric digits in the string;
- V. the positions of all vowels in the string. [P4.3]

04.1.3 Sides. Write a program that reads an integer n and displays, using asterisks (*), a full square and a rhombus, whose side has length n . For example, if the user provides the number 4, the program should display:

```

****
****
****
****

  *
 ***
*****
*****
*****
 ***
  *

```

[P4.22]

04.1.4 Words in reverse. Write a program that reads a word and displays:

- I. the word backwards. If, for example, the user provides the string 'Hello', the program should display 'olleH' [P4.9]
- II. uppercase letters starting from the end. For example, if the user provides the string 'CiAo', the program should display 'AC'.

04.1.5 Prime numbers. Write a program that asks the user for an integer and displays a message saying whether the number is prime or not.

[SUGGESTED] 04.1.6 Prime numbers. Write a program that asks the user for an integer and then displays all prime numbers less than or equal to that number. For example, if the user provides the number 20, the program should display:

```

2
3
5
7
11
13
17
19

```

[P4.17]

04.1.7 Words in pieces. Write a program that reads a word and displays all its substrings, sorted by increasing length. For example, if the user provides the string 'rum', the program should display:

```

r
u
m
ru
um
rum

```

[P4.12]

04.1.8 Duplicate numbers. Write a program that reads a sequence of integers (the sequence ends when an empty string is inserted) and that, after each new acquisition, finds and displays only the duplicate adjacent numbers.

The adjacent numbers repeated at least twice should be printed, **at the end of the sequence**, i.e., when the first different number is acquired.

If, for example, the input values at step IV are:

1
3
3
4
5
5
6
6
6
3

the program will display:

- I. at the first acquisition (1), nothing, because the value considered is only one;
- II. at the second acquisition (3), nothing, because the two adjacent values considered (1 and 3) are not duplicates, i.e. equal to each other;
- III. at the third acquisition (3), still nothing, because the acquired value is the duplicate of the previous value, so the sequence of duplicate adjacent numbers has begun, but it is not certain that it has ended;
- IV. At the fourth acquisition (4), the value 3, because the two adjacent values considered (3 and 4) are not duplicates; therefore, the sequence of duplicate adjacent numbers is finished, and the program should display the duplicate number.

With this logic, the program will also print 5 and 6 as soon as a different number is acquired.
[P4.2]

Part 2 – Applications of Cycles

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least one exercise during the lab session, and the remaining ones at home.

04.2.1 Nim's Game. In this game, two players alternately take marbles from a pile. With each move, a player chooses how many marbles to take: at least one and at most half of the available marbles. The player who takes the last marble loses the game.

Write a program that allows the user to play against the computer. The programme must:

- I. generate a random integer between 10 and 100 to be used as the initial size of the pile of marbles;
- II. generate a random integer, either 0 or 1, used to decide whether the user or the computer will play first;
- III. Generate a random integer, either 0 or 1, used to decide whether the computer will play smart or stupid:
 - a. playing stupidly, at each of its moves the computer simply takes a random number of marbles from the pile (the number should be valid, i.e. between 1 and $n/2$, if there are n marbles left in the pile);

- b. playing in intelligent mode, on the other hand, it draws a number of marbles such that the number of those remaining in the pile is a power of two decreased by one unit, i.e. 3, 7, 15, 31 or 63. The latter is always a valid move, except when the size of the pile is equal to a power of two decreased by one unit. In this case, the computer makes a move chosen at random (obviously among the valid ones).

As you can verify experimentally, the computer cannot be beaten when playing in smart mode and making the first move, unless the initial pile size is 15, 31 or 63. Similarly, a human player who makes the first move and knows the strategy described here is able to beat the computer. [P4.23]

04.2.2 Diagnostic imaging. The decay of radioactive materials can be modeled by the equation $A = A_0 e^{-\lambda t}$, where A is the amount of material at time t , A_0 is the quantity at time 0, and λ is the rate of decay. In particular as $\lambda = \frac{\ln 2}{T_{1/2}}$, where $T_{1/2}$ is the half-life of the substance (expressed in the same unit of measurement of t).

Technetium-99 is a radioisotope that is used for imaging of the brain. It has a half-life of 6 hours. Write a program that displays the relative amount A/A_0 in a patient's body for each hour during the 24 hours after receiving the dose. [P4.39]

[SUGGESTED] 04.2.3 Trajectories. Suppose that a cannonball is projected into the air with an initial velocity v_0 . Any physics book will state that the position of the ball after t seconds is $s(t) = -\frac{1}{2}gt^2 + v_0t$, where $g = 9.81 \text{ m/s}^2$ is the acceleration of the Earth's gravity. However, no book mentions why anyone would intend to proceed with this clearly dangerous experiment; therefore, we will carry it out in total safety, with the help of the computer. In fact, we will confirm the theory with the help of a simulation. In our simulation, we will consider how the ball moves in very small time intervals Δt . In a small interval of time, the velocity v is practically constant, and we can calculate the distance traveled by the ball as $\Delta s = v\Delta t$. Write a program that sets the constant $\text{DELTA_T} = 0.01$ (seconds) and updates the position of the ball with:

$$s = s + v * \text{DELTA_T}.$$

The speed is constantly changing, as it is reduced by the gravitational force of the Earth. In a small period of time, $\Delta v = -g\Delta t$ therefore, we must keep the speed updated with:

$$v = v - g * \text{DELTA_T}.$$

In each subsequent iteration, the new velocity value is used to update the distance traveled calculation. The program must continue running the simulation until the cannonball falls to the ground. The program takes the initial velocity as input (for example, 100 meters per second). It updates the position and speed $1/\text{DELTA_T}$ times for each simulated second, but outputs the calculated position only once per simulated second, along with the value calculated by the exact formula $s(t) = -\frac{1}{2}gt^2 + v_0t$, for comparison.

Note: There may be some doubt as to why it is advantageous to simulate a phenomenon for which there is an exact formula. The reason is that the formula is not really exact. For example, the acceleration of gravity varies depending on the distance from Earth. This is not represented by the formula, while our simulation could be extended to include this aspect. This is why simulation is necessary for the trajectories of objects flying higher, such as missiles. [P4.37]